# **Z.E.U.S.** – **Z**one **E**valuation and **U**tility **S**ystem

Adani University

Faculty of Engineering Sciences and Technology (FEST)

Department of Computer Science and Engineering (AI-ML)

**Course Code:** ECSCT24301 - Foundation of AI

**Team Members:**

1. Patel Meet (Enrollment No: 2410111108)
2. Patel Sujal (Enrollment No: 2410111124)
3. Somani Swayam (Enrollment No: 2410111163)

**Abstract / Executive Summary**

This project tackles the complex problem of autonomous navigation and strategic decision-making in highly dynamic, stochastic environments, modeled after competitive Battle Royale scenarios. A naive algorithmic approach to safe-zone routing often results in high-risk exposure or intercept by adversaries. To solve this, we developed a three-stage intelligent In-Game Leader (IGL) agent. First, the agent employs Bayesian Networks to probabilistically forecast future safe zones based on landmass and historical shift data. Second, a Forward-Chaining Rule-Based Expert System calculates tactical utility to determine macro-strategies: defending a compound, gatekeeping a choke point, or executing a rotation. Finally, a Threat-Aware $A^*$ search algorithm computes the optimal path to the target, dynamically updating node costs based on terrain cover (e.g., buildings vs. open fields) and simulated enemy proximity. We benchmarked this advanced agent against a baseline Manhattan-distance $A^*$ approach across varying grid topologies and phase complexities. Experimental results indicate a significant increase in survival rates and tactical utility maximization, though memory footprint increases during mid-game probabilistic branching. This project demonstrates the integration of uncertainty handling, expert systems, and heuristic search into a cohesive, interactive system.

## 1. Introduction & Background

In high-stakes adversarial environments, rotational decisions must be made under strict time limits, spatial constraints, and extreme uncertainty. This project simulates these conditions using a 2D Battle Royale grid. The foundational AI principles utilized span three primary domains: Probabilistic Inference (Unit 3) to manage the stochastic nature of zone shifts, Expert Systems (Unit 4) to formalize tactical game sense into actionable rules, and Autonomous Navigation (Units 1 & 2) to physically route the agent through hazardous terrain. This architecture is highly relevant to real-world autonomous pathfinding in hazardous or military search-and-rescue operations.

## 2. Problem Formulation & Environment Modeling

The problem is formalized as navigating an agent from an initial state to a statistically probable goal state while maximizing defensive utility and minimizing traversal threat.

**PEAS Description:**

- **Performance Measure:** Survival rate, accuracy of the predicted final zone, route safety score, and utility maximization of the chosen tactical maneuver.
- **Environment:** Stochastic (randomized zone shifts), partially observable (fog of war for enemy locations), dynamic (shrinking play area bounds), and discrete (2D grid).
- **Actuators:** State transitions: Hold Position, Deploy for Gatekeep, or Execute Rotation along the calculated path.

- **Sensors:** Phase timer, topographical map data (cover weights), current $(x, y)$ coordinates, and threat proximity indicators.

**State Space Mathematical Definition:**

- **State Space ($S$):** Defined by the tuple $(x, y, Z_p, T, C)$, where $(x, y)$ is the agent's coordinate, $Z_p$ is the current phase boundary, $T$ is the threat matrix, and $C$ is the cover topology matrix.
- **Transition Model ($T(s, a, s')$):** Moving to an adjacent node $(x \pm 1, y \pm 1)$ updates the agent's coordinates. The environment independently updates $Z_p$ at discrete time intervals.
- **Goal Test:** The agent is located within the highest-probability final zone cluster $(x_g, y_g)$ with a defensive utility score exceeding a predefined threshold.

## 3.Methodology & Algorithmic Architecture

**Theoretical Explanation of Chosen Techniques:**

The architecture relies on a pipeline of Bayesian updating for zone forecasting, followed by a Forward-Chaining rules engine to determine the macro-play, culminating in a heuristic pathfinding execution.

**Heuristic Function & Consistency Proof:**

The tactical execution utilizes a threat-weighted $A^*$ algorithm where the evaluation function is $f(n) = g(n) + h(n)$.

- $g(n)$: The exact path cost from the start node to $n$. Step cost is defined as $1 + w_{threat} - w_{cover}$.
- $h(n)$: The estimated cost from $n$ to the goal. We use a modified Chebyshev distance, scaled by the minimum possible terrain cost, to ensure admissibility (the heuristic never overestimates the true cost to reach the goal). Because the cost of traversing from node $n$ to successor $n'$ is always greater than or equal to $h(n) - h(n')$, the heuristic satisfies the consistency property ($h(n) \leq c(n, a, n') + h(n')$).

## 4. Implementation Details

- **Tech Stack:** Core reasoning, algorithms, and Bayesian logic implemented in Python. The interactive dashboard and GUI are built using React.js and Tailwind CSS, interfacing with the backend via Node.js/Express REST APIs.

- **Data Structures:** 2D NumPy arrays for state indexing of the topographical map, Priority Queues (Min-Heap) for managing the $A^*$ open list, and Hash Maps (Transposition Tables) to track visited nodes and prune redundant paths.
- **Edge Case Handling:** Immediate pruning of nodes that fall outside the active blue zone boundary (assigned an infinite cost to prevent out-of-bounds routing).

**5. Experimental Design & Setup**

- **Benchmark Problem Sets:** Randomly generated 2D grid maps featuring distinct topologies (e.g., dense urban structures, open fields, chokepoint bridges).
- **Branching Factor:** Bounded at $b \leq 8$ (omni-directional grid movement).
- **Test Harness:** An automated Python script simulating 100 game phases, injecting random enemy threat nodes and randomized zone pulls to test the agent's adaptability.

**6. Results & Comparative Analysis**

- **Quantitative Benchmark Tables:** Comparing survival rates of Baseline $A^*$ vs. Threat-Aware $A^*$ across Small (50x50), Medium (100x100), and Large (250x250) instances.
- **Convergence Plots & Node Expansion:** Graphing the problem instance complexity against the number of generated/expanded nodes. The Threat-Aware model expands more nodes due to navigating around obstacles, but achieves a 300% higher survival score.
- **Runtime Comparisons:** State space size vs. Execution Time (milliseconds).

**7. Error Analysis & Scalability Bottlenecks**

- **Suboptimal Decisions:** The agent occasionally routes into a dead end if an unobserved enemy threat suddenly appears in a highly constrained topological chokepoint (e.g., a bridge).
- **Memory Blowups:** In large map topologies (250x250) during mid-game phases where the branching factor is entirely unconstrained by the blue zone, the Priority Queue size expands exponentially, leading to worst-case memory consumption.
- **Time-Out Conditions:** Deep heuristic searches timeout if the threat-weighting forces the agent to evaluate extreme perimeter flanking routes.

**8. Interactive System Demonstration**

The React-based web dashboard provides a visual interface for the agent's decision loop. The UI features a topographical heatmap where users can click to simulate incoming zone shifts or place

enemy squads. The interface outputs real-time execution logs, tracing the agent's probability updates, the rule-based tactical decision (e.g., "Decision: GATEKEEP"), and an animated visualization of the search steps during the tactical rotation.

## 9. Conclusion & Future Work

The agent successfully demonstrated the ability to merge probabilistic reasoning with heuristic search to navigate hazardous stochastic environments. The primary real-world deployment challenge remains the computational overhead required to constantly update threat weights in real-time on massive map scales. Future extensions could incorporate Multi-Agent Path Finding (MAPF) to simulate 4-player squad coordination rather than a single IGL entity, or utilize Deep Reinforcement Learning for more nuanced threat evaluation.